



UNIVERSITY OF LEEDS

COMP3011 Web Services and Web Data module

Coursework 1

Slimani Zakaria
201726008

Football Analytics API – Technical Report

Introduction

This report presents the design and implementation of the Football Analytics API, developed as part of the COMP3011 Web Services and Web Data module. The project involves building a RESTful web service that provides football-related data analytics alongside a simple frontend interface for interacting with the API.

The system enables users to explore football player data, perform analytical queries (e.g., top scorers, market values), and manage custom favourite player lists through a fully functional CRUD interface. The API is implemented using FastAPI, with a layered architecture that separates routing, business logic, and data access.

Links (for submission):

- **GitHub repository:** <https://github.com/Zakari21s/football-analytics-api>
- **API documentation:** [API Documentation \(PDF\)](#) in this repository. Interactive docs at /docs (Swagger UI) and /redoc when the server is running (or at *base-url/docs*).
- **Presentation slides:** [Football Analytics Presentation.pptx](#)

1. Technology Stack and Justification

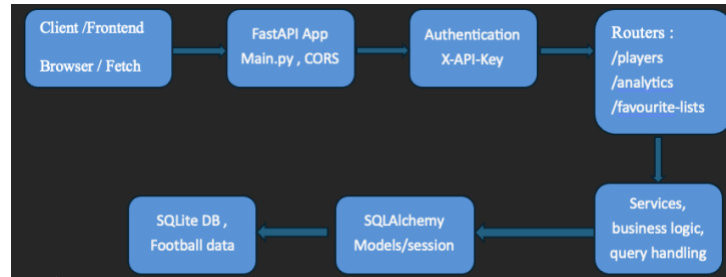
The Football Analytics API was developed using a combination of modern web technologies selected for their simplicity, efficiency, and suitability for RESTful API development. The chosen stack supports rapid development, clear structure, and ease of testing, while remaining appropriate for the scope of the coursework project.

Layer	Technology	Justification
Backend	Python 3.10+, FastAPI	Provides high performance with asynchronous support, automatic OpenAPI documentation (Swagger/ReDoc), and built-in validation, making it well suited for REST API development.
ORM	SQLAlchemy 2.x	Enables structured interaction with relational data using models and relationships, supporting complex queries required for football datasets.
Validation	Pydantic	Ensures type-safe request and response validation, producing consistent API outputs and clear error handling.
Database	SQLite	Lightweight, file-based database requiring no server setup, suitable for development and small-scale deployment in coursework projects.
Testing	pytest, FastAPI TestClient	Allows efficient testing of API endpoints, including status codes, validation, and JSON responses without running a live server.
Frontend	HTML, CSS, JavaScript	Provides a lightweight interface for interacting with the API using fetch requests, sufficient for demonstrating functionality within project scope.

This technology stack aligns with the principles of web service design introduced in the COMP3011 Web Services and Web Data lectures (University of Leeds, 2025).

2. Architecture

The Football Analytics API follows a layered architecture that separates client interaction, request handling, business logic, and data access. This improves maintainability and ensures clear separation of concerns.



The architecture begins at the client (frontend), which sends HTTP requests to the API using JavaScript. These requests are handled by the **FastAPI application**, which acts as the system's entry point and routes requests to the appropriate endpoints.

Before accessing protected routes, requests pass through an **authentication layer**, where the X-API-Key is validated to ensure authorised access.

The **router layer** defines API endpoints for resources such as players, analytics, and favourite lists. Routers are kept thin and delegate processing to the **service layer**, which contains the core business logic, including data processing and analytics computations.

The **SQLAlchemy layer** manages communication with the database using models and sessions, enabling structured interaction with relational data.

Finally, the **SQLite database** stores the football dataset and application data, providing persistent storage.

3. Design Choices

The design of the Football Analytics API follows RESTful principles, focusing on simplicity, consistency, and usability. Key decisions were made regarding API structure, resource modelling, authentication, and data handling to ensure a maintainable and scalable system.

RESTful Design

The API is structured using resource-based endpoints under a versioned prefix (`/api/v1/`), supporting future extensibility. For example, endpoints such as `GET /players/` and `POST /favourite-lists/` follow standard REST conventions. This approach improves clarity and follows REST principles from the COMP3011 lectures (University of Leeds, 2025).

Status Codes and Error Handling

Standard HTTP status codes are used to represent request outcomes, including 200 OK, 201 Created, 204 No Content, 401 Unauthorized, and 404 Not Found. Error responses follow a consistent JSON structure, improving client-side handling and debugging.

Resource Design

The API distinguishes between full CRUD and partial resources. The **FavouriteList** resource supports full CRUD operations, while **FavouriteListPlayer** is treated as a relationship resource, supporting only add, remove, and view operations. This reduces unnecessary complexity.

Pagination and Filtering

Pagination is implemented using query parameters such as page and limit, with responses including metadata (e.g., total pages and count). Sorting and filtering options are also supported, allowing flexible data retrieval without increasing the number of endpoints.

Authentication

A simple API key mechanism (X-API-Key) is used to protect all /api/v1/* endpoints. This approach is sufficient for coursework scope and integrates well with FastAPI's dependency system, ensuring consistent authentication across the API.

Design Trade-offs

SQLite was chosen for simplicity and ease of deployment, although a database such as PostgreSQL would be more suitable for larger-scale systems. Similarly, a single endpoint was used for player details to simplify frontend integration, at the cost of larger response sizes. REST was selected over GraphQL due to its simplicity and alignment with project requirements.

4. Implementation Highlights

This section highlights key implementation aspects of the API, focusing on data processing, query handling, and analytical functionality.

Sorting and Aggregation

The API supports flexible sorting using query parameters such as sort_by and order. Fields such as **age**, **market value**, and **minutes played** are dynamically derived rather than directly stored.

For example, age is calculated from date_of_birth, while total minutes and goals are computed using aggregation queries over the player_performances table. Market value is determined using the most recent entry per player from the player_market_value dataset.

Analytics Endpoints

Several analytical endpoints were implemented to extract insights from the dataset:

- **Top scorers:** total goals per player using aggregation
- **Top assists:** total assists per player
- **Most minutes played:** total minutes per player
- **Top market values:** latest market value per player

These endpoints support optional filters such as season or competition, allowing more targeted queries. All analytics are computed at request time, ensuring up-to-date results without requiring precomputed data.

Player Details Endpoint

A dedicated endpoint (/players/{id}/details/) was implemented to provide a comprehensive view of a player. This includes:

- Current market value (latest entry)
- Market value history (time-series data)
- Career summary (seasons played and previous clubs)

Combining these elements into a single response simplifies frontend integration, as all relevant information can be retrieved with one request.

Data Processing Pipeline

The dataset was pre-processed before being loaded into the database. Data was filtered to include only relevant leagues, and CSV files were imported using scripts that respect foreign key relationships (e.g. teams before players).

To improve efficiency, data loading was performed using batched inserts and streaming techniques, avoiding loading entire files into memory.

Design Considerations

The implementation prioritises simplicity and clarity while still demonstrating advanced functionality such as aggregation, filtering, and relational queries. While analytics are computed at request time for flexibility, this may introduce performance limitations at scale. In larger systems, techniques such as caching or materialised views could be used to optimise performance.

5. Testing Approach

Testing was carried out using **pytest** and FastAPI's **TestClient**, allowing API endpoints to be tested without running a live server. This approach enabled efficient validation of request handling, response structure, and error conditions.

The testing strategy focused on key functional areas:

- **Authentication**: verifying that requests without a valid API key return 401 Unauthorized, while valid requests succeed
- **CRUD operations**: testing the full lifecycle of favourite lists (create, retrieve, update, delete), including handling of invalid IDs
- **Relationship operations**: ensuring correct behaviour when adding or removing players from favourite lists, including prevention of duplicates (409 Conflict)
- **Validation**: confirming that invalid inputs (e.g. missing fields) return appropriate error responses (422 Unprocessable Entity)
- **Data retrieval**: verifying that endpoints such as players and analytics return the expected structure and status codes

Tests were executed using `pytest tests/ -v`, and a shared configuration provided reusable components such as a test client and authentication headers.

Overall, this testing approach ensures that core API functionality is reliable and behaves as expected under both normal and edge-case conditions.

6. Deployment

The API was deployed using PythonAnywhere, making it accessible through a public URL. The application is hosted with the same codebase used in local development, ensuring consistency between environments.

Environment variables such as the database path and API key were configured on the host system. The SQLite database was initialised and populated using dedicated scripts before deployment.

The API documentation is available through Swagger UI and ReDoc, allowing interactive testing of endpoints. The frontend is served through the same application, enabling direct interaction with the API.

- **Live URL:** <https://zakari21s.pythonanywhere.com>
- **Frontend:** <https://zakari21s.pythonanywhere.com/app/>
- **Docs:** <https://zakari21s.pythonanywhere.com/docs> and <https://zakari21s.pythonanywhere.com/redoc>

7. Use of Generative AI (see Appendix A)

Generative AI tools were used throughout this project to support architecture design, endpoint planning, implementation, testing, and documentation. AI outputs were treated as suggestions rather than final answers: each proposed change was reviewed, adapted to project requirements, and validated through manual checks and automated tests.

This approach enabled faster iteration while maintaining technical understanding and ownership of decisions. In particular, AI support was used to explore alternative API designs, refine query logic for analytics endpoints, and improve consistency in validation and error handling.

A full declaration of tools used, purposes, representative interaction excerpts, and reflective analysis is provided in **Appendix A**.

8. Limitations and Future Work

The current system has several limitations. Authentication is based on a single API key, without user-specific access control or rate limiting. Additionally, SQLite is suitable for small-scale use but does not support high concurrency or large-scale deployment.

Analytics are computed at request time, which may impact performance for larger datasets. Future improvements could include caching or precomputed aggregates to optimise performance.

Further enhancements could involve implementing a more advanced authentication system, migrating to a scalable database such as PostgreSQL, and improving the frontend with better user experience and responsiveness.

9. Conclusion

In conclusion, the Football Analytics API demonstrates the design and implementation of a RESTful web service using modern technologies. The system integrates data processing, analytics, and user-driven functionality through a structured and modular architecture.

The project applies key software engineering principles, including separation of concerns, consistent API design, and effective testing. While the current implementation meets the coursework requirements, it also provides a foundation for future extensions and improvements.

Appendix A: Generative AI Declaration

A.1 Tools used and purpose

The following Generative AI tools were used during this project:

Cursor (AI-assisted development environment): used for planning architecture, refining API structure, improving code clarity, and drafting technical documentation.

Claude/Codex models via Cursor: used to generate and review candidate implementations for FastAPI routes, SQLAlchemy queries, Pydantic schemas, test cases, and frontend integration patterns.

Other GenAI tools: none.

All AI use is declared in this appendix. No undeclared AI tools were used.

A.2 Representative conversation examples

The examples below summarise representative interactions. They demonstrate how AI support was used and how outputs were reviewed before adoption.

Example 1 – Authentication design

Task: enforce authentication across all API v1 endpoints.

Prompt summary: asked how to apply API key checks consistently in FastAPI.

AI suggestion summary: use a shared dependency (e.g., `require_api_key`) and attach it at router level for `/api/v1`.

Author action: implemented dependency-based validation of X-API-Key and applied it to protected routes.

Verification: confirmed unauthorized requests return 401 and authorized requests succeed.

Example 2 – Player details endpoint

Task: provide richer player information in one request.

Prompt summary: asked how to combine current market value, history, and career summary.

AI suggestion summary: query latest market-value row, include ordered historical values, and aggregate career metadata.

Author action: implemented `/players/{id}/details` using service + schema layers.

Verification: checked response structure and not-found behavior.

Example 3 – Analytics endpoint extension

Task: add additional analytics beyond top scorers.

Prompt summary: requested help designing top-assists and youngest-stars endpoints with filters.

AI suggestion summary: use grouped SQLAlchemy queries with optional season/competition filters and age-derived constraints.

Author action: implemented new endpoints with consistent query parameters and response models.

Verification: validated status codes, schema shape, and filter behavior via tests/manual calls.

Example 4 – Frontend/API integration issues

Task: resolve request failures caused by URL/CORS mismatch.

Prompt summary: described redirect/preflight issues and asked for corrective approach.

AI suggestion summary: keep endpoint URL patterns consistent and align frontend calls with backend route style; adjust CORS policy accordingly.

Author action: updated request paths and configuration to ensure consistent behavior.

Verification: retested frontend actions (including POST operations) in local/deployed environments.

Example 5 – Testing strategy

Task: improve confidence in API behavior.

Prompt summary: asked what tests should be added for CRUD, validation, and analytics.

AI suggestion summary: include lifecycle tests, error-path tests (404, 409, 422, 401), and response-shape assertions.

Author action: added/extended pytest coverage for core flows.

Verification: executed test suite and reviewed outcomes.

A.3 Reflection and analysis of GenAI use

GenAI was used in a methodical and supervised way. Outputs were treated as draft suggestions, not final truth. Each suggestion was evaluated against coursework requirements, API design principles, and project constraints before being integrated.

The main benefits were:

faster iteration on architecture and endpoint design;
improved consistency in response formats, validation, and error handling;
broader exploration of alternatives (e.g., endpoint shapes, query design, and integration options).
The main limitations were:

occasional generic recommendations requiring domain-specific adaptation;
need for careful verification to avoid incorrect assumptions in generated code/text;
extra review time to ensure maintainability and correctness.
Overall, GenAI improved productivity and ideation, while final technical judgement, implementation choices, and validation remained the author's responsibility.

A.4 Supplementary evidence

Exported conversation logs and representative excerpts are provided as supplementary material accompanying this submission. These records document the interactions summarised above and provide evidence of declared GenAI usage in accordance with module guidance.

Reference List:

1. Alsalka, M. (2025) COMP3011 Web Services and Web Data: Coursework 1 Assessment Brief. University of Leeds
2. FastAPI (2026) *FastAPI documentation*. Available at: <https://fastapi.tiangolo.com/> (Accessed: 17 March 2026).
3. Kaggle (n.d.) *Football datasets (Transfermarkt-derived)*. Available at: <https://www.kaggle.com/datasets/xfkzujqjvx97n/football-datasets/code> (Accessed: 17 March 2026).
4. Pydantic (2026) *Pydantic documentation*. Available at: <https://docs.pydantic.dev/> (Accessed: 17 March 2026).
5. pytest (2026) *pytest documentation*. Available at: <https://docs.pytest.org/> (Accessed: 17 March 2026).
6. PythonAnywhere (2026) *PythonAnywhere documentation*. Available at: <https://help.pythonanywhere.com/> (Accessed: 17 March 2026).
7. SQLAlchemy (2026) *SQLAlchemy documentation*. Available at: <https://docs.sqlalchemy.org/> (Accessed: 17 March 2026).